

Upwork Discord Bot

Comprehensive Development Guide

Master Development Plan & Evaluation Guide

Language: Python 3.8+

Duration: 1 Week Development + 100 Hours Testing

Level: Advanced

A Production-Ready System Development Challenge

Generated on January 13, 2026

Upwork Discord Bot - Comprehensive Development Guide

Table of Contents

1. Project Overview
 2. What You'll Learn
 3. Technical Foundation
 4. Core Concepts Explained
 5. Development Phases
 6. Production Requirements
 7. Evaluation Criteria
 8. Success Guidelines
-

Project Overview

Mission

Build a production-grade Discord bot that **automatically scrapes and posts job listings from Upwork** to designated Discord channels. This project evaluates your ability to:

- Reverse engineer web applications
- Handle authentication and session management
- Build resilient, long-running systems
- Think like a production engineer

Example of Running Bot

<https://www.loom.com/share/0b812ceecf0241eeb004a129c3044fe3?sid=5589aca8-7b90-4ccb-9b91-49c6ae4e5cea>

Duration & Scope

- **Timeline:** 1 week development + 100 hours stability testing
- **Language:** **Python** (Required)
- **Complexity:** Advanced (requires problem-solving, not just coding)
- **Real-World Application:** This simulates actual production challenges you'll face

Critical Constraint: Network Requirements

■ **IMPORTANT:** When attempting this project, you **cannot use Bitbash Internet**. You **must** have:

- Your own mobile hotspot internet connection, OR
 - VPN with Bitbash internet (if you don't have personal internet)
-

What Youll Learn

Technical Skills

1. **Reverse Engineering:** Analyzing network traffic to understand undocumented APIs
2. **GraphQL:** Working with modern API query languages
3. **Browser Fingerprinting:** Understanding how websites identify and track clients
4. **Authentication Management:** Handling tokens, sessions, and credentials
5. **Asynchronous Programming:** Building concurrent, non-blocking systems
6. **Production Engineering:** Creating systems that run reliably for days

Problem-Solving Skills

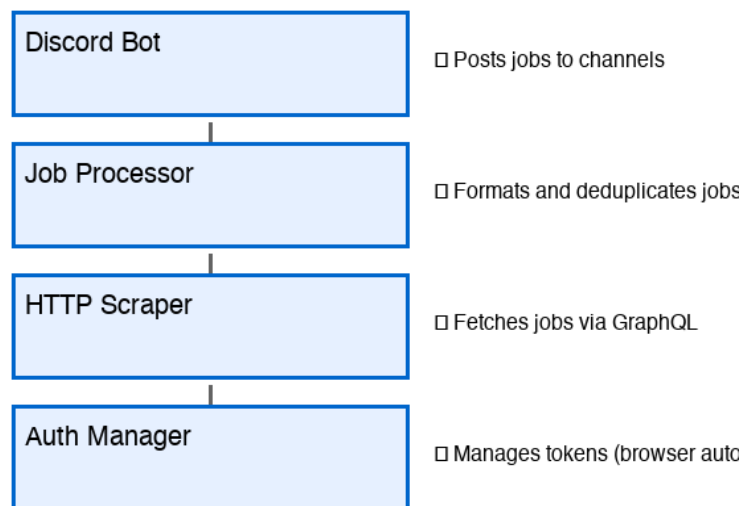
- How to approach problems when documentation doesn't exist
 - Debugging complex, multi-layered systems
 - Building resilient systems that handle failures gracefully
 - Balancing automation with security considerations
-

Technical Foundation

Required Stack (Python)

Language: Python 3.8+
HTTP Library: requests, httpx, or aiohttp
Discord Library: discord.py
Browser Automation (for auth ONLY): selenium, playwright, or seleniumbase
Database: SQLite or PostgreSQL (for tracking seen jobs)
Environment Management: python-dotenv

Architecture Overview



What You MUST Figure Out

- How to intercept and replicate Upwork's API calls
- What headers and cookies are required
- How to structure GraphQL queries

- How to refresh authentication without manual intervention
 - How to maintain session consistency
-

Core Concepts Explained

1. Understanding GraphQL APIs

What is GraphQL

GraphQL is a **query language for APIs** that lets you request exactly what data you need. Unlike REST APIs where you hit different endpoints (e.g., `/api/jobs`, `/api/job/123`), GraphQL uses a **single endpoint** and you send **queries** to specify what you want.

REST vs GraphQL Comparison

```
REST API:
GET /api/jobs?q=python&page=1
GET /api/job/12345
GET /api/user/789

GraphQL API:
POST /api/graphql
Body: {
  query: "query { jobs(search: 'python') { id, title, budget } }"
}
```

Why Upwork Uses GraphQL

Modern web applications (like Upwork, GitHub, Shopify) use GraphQL because:

- **Efficiency:** Get all data in one request
- **Flexibility:** Request only what you need
- **Type Safety:** Structured data schema
- **Real-time Updates:** Built-in subscription support

How to Discover GraphQL Endpoints

When you browse Upwork jobs, look for:

```
Request URL: https://www.upwork.com/api/graphql/v1?alias=visitorJobSearch
Request Method: POST
Content-Type: application/json
```

The payload will look like:

```
{
  "alias": "visitorJobSearch",
  "query": "query VisitorJobSearch($input: VisitorJobSearchInput!) { ... }",
  "variables": {
    "input": {
      "query": "python developer",
      "pagination": { "offset": 0, "limit": 20 }
    }
  }
}
```

GraphQL Query Structure

```
query GetJobs($searchTerm: String!) {
  jobs(search: $searchTerm) {
    id # Job ID
    title # Job title
    description # Job description
    budget { # Nested object
      amount
      currency
    }
    skills # Array of strings
    client { # Nested object
      location
      totalSpent
      reviewScore
    }
  }
}
```

Your Task with GraphQL

1. **Intercept:** Capture the actual query Upwork sends
2. **Replicate:** Reconstruct it in your Python code
3. **Customize:** Modify parameters (search query, pagination)
4. **Parse:** Extract job data from the response

Hint: Look for queries with aliases like `visitorJobSearch`, `visitorJobDetails`, or similar visitor-related operations.

2. Understanding Browser Fingerprinting & Authentication

What is Browser Fingerprinting

Browser fingerprinting is a technique websites use to **identify and track visitors** based on unique characteristics of their browser and system. Think of it like a digital fingerprint - even without cookies, websites can recognize you.

Why Websites Use Fingerprinting

- **Bot Detection:** Distinguish real users from automated scripts
- **Security:** Prevent fraud and account takeover
- **Tracking:** Monitor user behavior across sessions
- **Rate Limiting:** Control API usage per unique client

Components of a Browser Fingerprint

1. HTTP Headers

Every request your browser sends includes headers that reveal information:

```
{
  "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)...",
  "Accept": "text/html,application/xhtml+xml,...",
  "Accept-Language": "en-US,en;q=0.9",
  "Accept-Encoding": "gzip, deflate, br",
  "Referer": "https://www.upwork.com/",
  "Connection": "keep-alive",
  "Upgrade-Insecure-Requests": "1",
  # ... many more
}
```

What they reveal:

- Operating system (Windows, macOS, Linux)
- Browser type and version (Chrome 120, Firefox 115)
- Language preferences
- Previous page you visited
- Compression support

2. Cookies

Cookies are small pieces of data stored by your browser:

```
{
  "session_id": "abc123xyz789",
  "visitor_id": "v-1234567890",
  "__cf_bm": "cloudflare_bot_management_token",
  "_ga": "google_analytics_id",
  "IR_13634": "interaction_recording_token"
}
```

Types of cookies:

- **Session Cookies:** Temporary, expire when browser closes
- **Persistent Cookies:** Stored long-term
- **Authentication Cookies:** Prove you're logged in
- **Tracking Cookies:** Monitor your behavior
- **Security Cookies:** Anti-bot protection (Cloudflare, etc.)

3. JavaScript-Based Fingerprinting

When JavaScript runs in your browser, websites can detect:

```
// Screen information
screen.width, screen.height, screen.colorDepth

// Timezone
Intl.DateTimeFormat().resolvedOptions().timeZone

// Installed fonts (via Canvas fingerprinting)
// WebGL capabilities (GPU information)
// Audio context fingerprinting
// Battery API
// ... and 50+ other signals
```

4. TLS/SSL Fingerprinting

Even the way your browser establishes secure connections is unique:

- TLS version (1.2, 1.3)
- Cipher suites supported
- Extension order
- Compression methods

How Upwork Uses Fingerprinting

When you visit Upwork, the site:

1. **Checks your headers** - Does it look like a real browser?
2. **Validates cookies** - Do you have valid session tokens?
3. **Runs JavaScript** - Can you execute modern JavaScript?
4. **Generates tokens** - Creates authentication tokens based on your fingerprint
5. **Links your session** - Associates your requests with a unique visitor ID

The Challenge for Scrapers

If your Python script sends requests like this:

```
import requests
response = requests.get("https://www.upwork.com/api/graphql/v1")
```

Upwork sees:

```
User-Agent: python-requests/2.31.0 # ■■ NOT A BROWSER!
Accept: */* # ■■ TOO GENERIC!
# Missing 30+ headers a real browser sends
# No cookies
# No JavaScript execution
```

Result: Request blocked or returns error

Your Solution Strategy

Phase 1: Observe a Real Browser

1. Open Chrome DevTools
2. Go to Network tab
3. Visit <https://www.upwork.com/nx/search/jobs/?q=python>
4. Find the GraphQL request
6. Analyze what a REAL request looks like

Phase 2: Replicate the Fingerprint

```
# Instead of bare requests, you need to:
import cloudscraper # Handles Cloudflare automatically
# OR
import requests

session = requests.Session()

# Add ALL the headers from DevTools
headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36...",
    "Accept": "application/json, text/plain, */*",
    "Accept-Language": "en-US,en;q=0.9",
    "Accept-Encoding": "gzip, deflate, br",
    "Referer": "https://www.upwork.com/nx/search/jobs/",
    "Origin": "https://www.upwork.com",
    "Connection": "keep-alive",
    "Sec-Fetch-Dest": "empty",
    "Sec-Fetch-Mode": "cors",
    "Sec-Fetch-Site": "same-origin",
    # ... copy ALL headers from DevTools
}

# Add ALL cookies from DevTools
cookies = {
    "oauth2_global_js_token": "your_token_here",
    "visitor_id": "your_visitor_id",
    # ... copy ALL cookies
```

```

}

response = session.post(
    "https://www.upwork.com/api/graphql/v1",
    headers=headers,
    cookies=cookies,
    json={"query": "..."}
)

```

Phase 3: Handle Dynamic Tokens

The Problem: Some cookies and headers **change** with every request or expire.

Examples:

```

# These change frequently:
"Authorization": "Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." # Expires in 12 hours
"__cf_bm": "gqBVQ8Ks4ZKF..." # Cloudflare bot management (30 min)
"X-Request-ID": "uuid-1234-5678" # Unique per request

```

Your Task:

- Identify which tokens are **static** (copy once)
- Identify which tokens are **dynamic** (must refresh)
- Implement **automatic refresh** before tokens expire

Authentication Token Lifecycle

```

[Browser Visit]
[Token Expires]

```

Hints for Token Management

Key Questions to Answer:

1. Where does Upwork store the auth token? (Cookie? Header? LocalStorage?)
2. How long is it valid? (Check JWT expiration if it's a JWT)
3. Can you refresh it without full login? (Check for refresh token endpoints)
4. What's the minimum automation needed? (Just visit homepage? Or full login?)

Investigation Steps:

1. Login to Upwork in Chrome
3. Note all cookies (especially oauth, visitor_id, session)
5. Make a GraphQL request (search for jobs)
6. Find the Authorization header in request headers
7. Decode the token (if JWT, use jwt.io)
8. Note the expiration time

9. Delete all cookies
10. Refresh page - what happens?
11. Which cookies get regenerated?

3. The Two-Part System: HTTP Scraping Browser Automation

Why This Hybrid Approach

Pure Browser Automation (Selenium/Playwright):

```
# This works but is SLOW and resource-heavy
from selenium import webdriver
driver = webdriver.Chrome()
driver.get("https://www.upwork.com/nx/search/jobs/?q=python")
# Wait for page load, JavaScript execution...
# Extract data from HTML
# Repeat every 5-10 seconds
```

Problems:

- Takes 5-10 seconds per request
- Uses 200-500 MB RAM per browser instance
- Can't run 100 hours without crashing
- ■ Easily detected as automation

Pure HTTP Requests:

```
# Fast and efficient
import requests
response = requests.post("https://www.upwork.com/api/graphql/v1", ...)
# 100-200ms per request!
```

Problems:

- Need valid authentication tokens
- Tokens expire every 12 hours
- Can't generate tokens without a browser

HYBRID SOLUTION (What you must build):

```
Main Loop (Pure HTTP):
Every 5-10 seconds:
Every 11 hours:
```

Implementation Architecture

```
class UpworkScraper:
    def __init__(self):
        self.session = requests.Session() # HTTP client
        self.auth_token = None
        self.cookies = {}
        self.headers = {}

    def fetch_jobs(self, search_query):
        """Fast HTTP-based job fetching"""
        response = self.session.post(
            "https://www.upwork.com/api/graphql/v1",
            headers=self.headers,
            cookies=self.cookies,
            json=self.build_graphql_query(search_query)
        )
        return response.json()

    def refresh_authentication(self):
        """Browser automation (slow but necessary)"""
        # Launch headless browser
        # Visit Upwork
        # Extract tokens from cookies/localStorage
        # Update self.auth_token, self.cookies, self.headers
        # Close browser
        pass

    def should_refresh_auth(self):
        """Check if tokens are about to expire"""
        # Check token timestamp
        # Return True if < 1 hour remaining
        pass
```

Development Phases

Phase 1: Reverse Engineering the API

Objectives

Understand how Upwork's job search works

Identify GraphQL endpoints and queries

Capture required headers and cookies

Successfully fetch jobs via HTTP requests

Tasks

1.1 Network Traffic Analysis

Tools: Chrome DevTools (Network tab)

Steps:

1. Open <https://www.upwork.com/nx/search/jobs/?q=web%20developer>
3. Filter by "Fetch/XHR" or search "graphql"
4. Find the request that loads jobs
5. Examine:
 - Request URL
 - Request Method (likely POST)
 - Request Headers (ALL of them)
 - Request Payload (GraphQL query)
 - Response (JSON structure)
7. Convert cURL to Python (use <https://curlconverter.com/>)

1.2 GraphQL Query Extraction

Questions to answer:

- What's the query alias? (e.g., "visitorJobSearch")
- What variables does it accept?
- What fields does it return?
- How is pagination handled?
- Are there multiple queries for different data?

Hint: Look for two separate queries:

1. Job search/listing (returns basic job info)
2. Job details (returns full description, client info)

1.3 Initial Python Implementation

```
# Your first working script:
import requests # or cloudscraper
import json

# Copy headers from DevTools
HEADERS = {
    # ... paste all headers here
}

# Copy cookies from DevTools
COOKIES = {
    # ... paste all cookies here
}

# Copy GraphQL query from DevTools
GRAPHQL_QUERY = {
    "query": """
    # ... paste query here
    """,
    "variables": {
        # ... paste variables here
    }
}
```

```
# Make request
response = requests.post(
    "https://www.upwork.com/api/graphql/v1?alias=visitorJobSearch",
    headers=HEADERS,
    cookies=COOKIES,
    json=GRAPHQL_QUERY
)

print(response.status_code)
print(json.dumps(response.json(), indent=2))
```

Success Criteria:

- Script returns 200 OK
- Response contains job listings
- Data includes: job ID, title, description, budget

Common Issues:

- **403 Forbidden:** Missing headers or invalid cookies
 - **401 Unauthorized:** Auth token expired
 - **Empty response:** Wrong query alias or variables
 - **Cloudflare challenge:** Need cloudscraper library
-

Phase 2: Discord Integration

Objectives

Create Discord bot

Post jobs to channels

Create automatic threads with full details

Format messages professionally

Tasks

2.1 Discord Bot Setup

```
# Required library
pip install discord.py

# Basic bot structure
import discord
from discord.ext import commands, tasks

intents = discord.Intents.default()
intents.message_content = True
```

```

bot = commands.Bot(command_prefix="!", intents=intents)

@bot.event
async def on_ready():
    print(f"Bot logged in as {bot.user}")
    job_scraper_loop.start() # Start your scraping loop

@tasks.loop(seconds=10) # Run every 10 seconds
async def job_scraper_loop():
    # Fetch jobs from Upwork
    # Post new jobs to Discord
    pass

bot.run("YOUR_DISCORD_TOKEN")

```

2.2 Job Posting Format

Requirements:

```

Title: {job_title}
Posted: {time_ago} # e.g., "30 minutes ago"
Budget/Rate: {amount} # e.g., "$500-$1000" or "$25/hr"
Level: {experience_level} # Entry/Intermediate/Expert
Time: {HH:MM} # 24-hour format when bot detected it
Proposals: {proposal_count}
Client Info: {payment_verified}, {location}, ${total_spent} spent

{description_preview} # First 200-300 characters

[Apply Here]({job_url})

```

Example Output:

```

■ New Job Posted!

Title: Build a Python Discord Bot
Posted: 15 minutes ago
Budget: $500-$1000
Level: Intermediate
Time: 14:35
Proposals: 2
Client Info: ■ Payment verified, United States, $15,234 spent

Looking for an experienced Python developer to build a Discord bot
that can scrape job listings and post them automatically. Must have
experience with async programming and...

[Apply Here](https://www.upwork.com/jobs/~abc123xyz)

```

2.3 Automatic Thread Creation

Requirement: Each job post must automatically create a thread containing full details.

```
# Discord.py example (hint)
message = await channel.send("Job post content...")

# Create thread automatically
thread = await message.create_thread(
    name=f"Job: {job_title[:80]}", # Thread title (max 100 chars)
    auto_archive_duration=60 # Archive after 60 minutes of inactivity
)

# Post full details in thread
await thread.send(f"""
**Full Job Description**:
{full_description}

**Client Details**:
- Total Spent: ${client_total_spent}
- Jobs Posted: {client_jobs_posted}
- Hire Rate: {client_hire_rate}%
- Location: {client_location}
- Member Since: {client_member_since}

**Job Details**:
- Duration: {project_duration}
- Experience Level: {experience_level}
- Job Type: {job_type}

[Apply on Upwork]({job_url})
""")
```

Hint: You'll need to make a **second GraphQL request** to get full job details (the search query only returns basic info).

Success Criteria:

- Jobs posted to correct channel
- Messages formatted correctly
- Threads created automatically (no manual clicks)
- Thread contains all required information
- Time format is 24-hour (HH:MM)
- Client spending displayed accurately

Phase 3: Authentication Management

Objectives

Implement automatic token refresh

Use browser automation for token extraction

System runs without manual intervention

Handle authentication failures gracefully

The Challenge

Upwork's authentication tokens expire approximately every 12 hours. Your bot must:

1. **Detect** when tokens are about to expire or have expired
2. **Automatically** refresh them without stopping the main loop
3. **Update** the HTTP client with new credentials
4. **Continue** fetching jobs seamlessly

Recommended Approach

Option A: Proactive Refresh (Recommended)

```
import time
from datetime import datetime, timedelta

class AuthManager:
    def __init__(self):
        self.token_timestamp = None
        self.token_lifetime = timedelta(hours=11) # Refresh before 12h expiration

    def should_refresh(self):
        if not self.token_timestamp:
            return True

        elapsed = datetime.now() - self.token_timestamp
        return elapsed >= self.token_lifetime

    def refresh_tokens(self):
        """Use browser automation to get fresh tokens"""
        # Launch browser
        # Login or visit Upwork
        # Extract tokens
        # Update timestamp
        self.token_timestamp = datetime.now()
```

Option B: Reactive Refresh

```
def fetch_jobs_with_auto_retry(search_query):
    try:
        response = make_graphql_request(search_query)
        if response.status_code == 401: # Unauthorized
            print("Token expired, refreshing...")
            refresh_tokens()
        response = make_graphql_request(search_query) # Retry
        return response
    except Exception as e:
```

```
print(f"Error: {e}")
return None
```

Browser Automation for Token Extraction

Critical Rule: You can ONLY use browser automation for **authentication**, NOT for fetching jobs.

Allowed:

```
# Selenium/Playwright/SeleniumBase for token refresh
from selenium import webdriver

def refresh_auth_tokens():
    driver = webdriver.Chrome(options=headless_options)
    driver.get("https://www.upwork.com/")

    # Extract cookies
    cookies = driver.get_cookies()

    # Extract localStorage tokens
    token = driver.execute_script("return localStorage.getItem('oauth2_access_token')")

    driver.quit()
    return cookies, token
```

NOT Allowed:

```
# ■ Using browser to scrape jobs (too slow, violates requirements)
driver.get("https://www.upwork.com/nx/search/jobs/?q=python")
jobs = driver.find_elements(By.CLASS_NAME, "job-tile") # NO!
```

Implementation Hints

Where to Find Tokens:

```
# After logging in with Selenium:

# 1. Check cookies
cookies = driver.get_cookies()
for cookie in cookies:
    if 'oauth' in cookie['name'] or 'token' in cookie['name']:
        print(f"Found: {cookie['name']} = {cookie['value']}")

# 2. Check localStorage
token = driver.execute_script("""
    return {
        'oauth_token': localStorage.getItem('oauth2_global_js_token'),
        'visitor_id': localStorage.getItem('visitorId'),
        'user_id': localStorage.getItem('userId')
    }
""")
```

```
# 3. Check sessionStorage
session_data = driver.execute_script("return sessionStorage")

# 4. Check request headers (intercept network)
# Use Selenium wire or Playwright to capture requests
```

Headless Browser Configuration:

```
from selenium import webdriver
from selenium.webdriver.chrome.options import Options

options = Options()
options.add_argument("--headless") # Run without GUI
options.add_argument("--no-sandbox")
options.add_argument("--disable-dev-shm-usage")
options.add_argument("--disable-gpu")
options.add_argument("--window-size=1920,1080")
options.add_argument("user-agent=Mozilla/5.0 (Windows NT 10.0; Win64; x64)...")

driver = webdriver.Chrome(options=options)
```

Login Automation (if required):

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

def automated_login(driver, email, password):
    driver.get("https://www.upwork.com/ab/account-security/login")

    # Wait for email field
    email_field = WebDriverWait(driver, 10).until(
        EC.presence_of_element_located((By.ID, "login_username"))
    )
    email_field.send_keys(email)

    # Click continue
    continue_button = driver.find_element(By.ID, "login_password_continue")
    continue_button.click()

    # Wait for password field
    password_field = WebDriverWait(driver, 10).until(
        EC.presence_of_element_located((By.ID, "login_password"))
    )
    password_field.send_keys(password)

    # Submit
    login_button = driver.find_element(By.ID, "login_control_continue")
    login_button.click()

    # Wait for redirect to dashboard
    WebDriverWait(driver, 20).until(
        EC.url_contains("upwork.com/nx/")
    )
```

```
# Now extract tokens
return extract_tokens(driver)
```

Success Criteria:

- Tokens refresh automatically every 11 hours
 - Browser automation runs headlessly (no GUI)
 - Main job scraping continues during refresh
 - Failed refresh attempts trigger retry logic
 - No manual intervention required
-

Phase 4: Multi-URL Support

Objectives

Track multiple Upwork search URLs simultaneously

Route jobs to correct Discord channels

Prevent job cross-contamination

Maintain accuracy across all URLs

Configuration Structure

config.json (or similar):

```
{
  "tracked_urls": [
    {
      "url": "https://www.upwork.com/nx/search/jobs/?q=python%20developer",
      "channel_id": 123456789012345678,
      "label": "Python Jobs"
    },
    {
      "url": "https://www.upwork.com/nx/search/jobs/?q=web%20scraping",
      "channel_id": 987654321098765432,
      "label": "Scraping Jobs"
    }
  ],
  "fetch_interval": 10,
  "token_refresh_hours": 11
}
```

Implementation Strategy

Approach 1: Sequential Processing

```

async def scrape_all_urls():
    for url_config in tracked_urls:
        jobs = fetch_jobs(url_config['url'])
        for job in jobs:
            if is_new_job(job['id'], url_config['url']):
                await post_to_discord(job, url_config['channel_id'])
            await asyncio.sleep(2) # Pause between URLs

```

Approach 2: Concurrent Processing (Advanced)

```

async def scrape_all_urls():
    tasks = []
    for url_config in tracked_urls:
        task = asyncio.create_task(scrape_single_url(url_config))
        tasks.append(task)
    await asyncio.gather(*tasks)

async def scrape_single_url(url_config):
    jobs = await fetch_jobs_async(url_config['url'])
    # Process jobs...

```

Deduplication Strategy

Problem: Same job might appear in multiple searches if it matches different keywords.

Solution: Track job IDs per URL

```

# Database schema
CREATE TABLE seen_jobs (
    job_id VARCHAR(255),
    url_source VARCHAR(500),
    first_seen TIMESTAMP,
    PRIMARY KEY (job_id, url_source)
);

# Python logic
def is_new_job(job_id, url_source):
    # Check if this specific job_id from this URL has been posted
    result = db.query(
        "SELECT 1 FROM seen_jobs WHERE job_id = ? AND url_source = ?",
        (job_id, url_source)
    )
    return result is None

def mark_job_as_seen(job_id, url_source):
    db.execute(
        "INSERT INTO seen_jobs (job_id, url_source, first_seen) VALUES (?, ?, ?)",
        (job_id, url_source, datetime.now())
    )

```

Success Criteria:

- Each URL checked every 5-10 seconds
 - Jobs posted to correct channel
 - No duplicate posts within same URL
 - Same job can appear in different channels if it matches multiple searches
-

Phase 5: Production Hardening

Objectives

Implement comprehensive error handling

Add logging and monitoring

Optimize resource usage

Prepare for 100-hour stability test

Error Handling Patterns

Network Errors:

```
import time
from requests.exceptions import RequestException

def fetch_jobs_with_retry(url, max_retries=3):
    for attempt in range(max_retries):
        try:
            response = requests.post(url, ...)
            response.raise_for_status()
            return response.json()
        except RequestException as e:
            print(f"Attempt {attempt + 1} failed: {e}")
            if attempt < max_retries - 1:
                time.sleep(2 ** attempt) # Exponential backoff
            else:
                raise
```

Discord API Errors:

```
from discord.errors import HTTPException

async def post_job_safely(channel, content):
    try:
        message = await channel.send(content)
        return message
    except HTTPException as e:
        if e.status == 429: # Rate limited
            print("Rate limited, waiting...")
            await asyncio.sleep(e.retry_after)
        return await post_job_safely(channel, content)
```

```

else:
    print(f"Discord error: {e}")
    return None

```

Logging System

```

import logging

# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s [%(levelname)s] %(message)s',
    handlers=[
        logging.FileHandler('bot.log'),
        logging.StreamHandler()
    ]
)

logger = logging.getLogger(__name__)

# Usage
logger.info("Bot started")
logger.warning("Token expiring soon")
logger.error("Failed to fetch jobs", exc_info=True)

```

Resource Management

Memory Monitoring:

```

import psutil
import os

def check_memory_usage():
    process = psutil.Process(os.getpid())
    memory_mb = process.memory_info().rss / 1024 / 1024
    logger.info(f"Memory usage: {memory_mb:.2f} MB")

    if memory_mb > 500: # Alert if over 500 MB
        logger.warning("High memory usage detected")

```

Database Cleanup:

```

def cleanup_old_jobs():
    """Remove seen jobs older than 30 days"""
    db.execute("""
DELETE FROM seen_jobs
WHERE first_seen < datetime('now', '-30 days')
""")

```

Graceful Shutdown

```
import signal
import sys

def signal_handler(sig, frame):
    logger.info("Shutdown signal received")
    # Save state
    save_current_state()
    # Close database connections
    db.close()
    # Stop Discord bot
    bot.close()
    sys.exit(0)

signal.signal(signal.SIGINT, signal_handler)
signal.signal(signal.SIGTERM, signal_handler)
```

Production Requirements

The 100-Hour Stability Test

Test Conditions

- **Duration:** 100 consecutive hours (4+ days)
- **Intervention:** Zero manual actions allowed
- **Monitoring:** Automated checks only
- **Success Metric:** Zero crashes, continuous operation

What Will Be Tested

1. Token Refresh Reliability

- Does it refresh automatically?
- Does it handle refresh failures?
- Does scraping continue during refresh?

2. Memory Management

- Does memory usage stay stable?
- Are there memory leaks?
- Does it handle long-running processes?

3. Error Recovery

- Network timeouts
- Discord API rate limits
- Database connection issues
- Upwork API changes

4. Data Accuracy

- Jobs posted to correct channels
- No duplicate posts
- All job details accurate
- Thread creation works consistently

Monitoring Dashboard (Optional but Recommended)

```
# Simple status endpoint
from flask import Flask, jsonify

status_app = Flask(__name__)

@status_app.route('/status')
def status():
    return jsonify({
        'uptime_hours': calculate_uptime(),
        'jobs_posted_last_hour': count_recent_jobs(),
        'last_token_refresh': last_refresh_time,
        'memory_usage_mb': get_memory_usage(),
        'active_urls': len(tracked_urls),
        'errors_last_hour': count_recent_errors()
    })

# Run in separate thread
import threading
threading.Thread(target=lambda: status_app.run(port=5000)).start()
```

Evaluation Criteria

Minimum Passing Requirements (Must Have All)

- Jobs fetched via HTTP requests (no browser for scraping)
- GraphQL API used correctly
- Posted to Discord with correct formatting

- Automatic thread creation with full details
- Automatic token refresh (browser automation OK)
- Multi-URL support working
- 100-hour test passed with zero crashes
- Complete documentation provided
- Code is clean and well-organized

Excellence Indicators (Stand Out)

- Innovative token refresh approach
- Exceptional error handling
- Resource-efficient implementation
- Comprehensive logging system
- Clean, modular code architecture
- Detailed documentation with examples
- Professional demo video
- Proactive communication during development

Automatic Failure (Any One of These)

- Using browser automation to scrape jobs
 - Manual token refresh required
 - Jobs posted to wrong channels
 - Bot crashes during 100-hour test
 - Duplicate jobs posted repeatedly
 - Missing critical documentation
 - Hardcoded credentials in code
 - Unable to explain implementation decisions
-

Documentation Requirements

User Manual Must Include:

1. Introduction

- What the bot does
- Key features
- Benefits

2. Setup Instructions

Prerequisites:

```
Python 3.8+
Discord Bot Token
Upwork Account (for token extraction)
PostgreSQL or SQLite (for job tracking)
```

Installation:

```
# Clone repository
git clone <repo-url>
cd upwork-discord-bot

# Create virtual environment
python -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate

# Install dependencies
pip install -r requirements.txt

# Configure environment
cp .env.example .env
# Edit .env with your credentials
```

Environment Variables:

```
DISCORD_TOKEN=your_discord_bot_token
CHANNEL_ID_PYTHON=123456789
CHANNEL_ID_SCRAPING=987654321
UPWORK_EMAIL=your@email.com
UPWORK_PASSWORD=your_password
DATABASE_URL=sqlite:///jobs.db
```

3. Configuration Guide

- Adding/removing tracked URLs
- Channel mapping
- Adjusting fetch intervals
- Token refresh timing

4. Features Documentation

- How job scraping works
- Discord posting format
- Token refresh mechanism
- Error handling

5. Troubleshooting

Common Issues:

Problem: "401 Unauthorized" errors
Solution: Run token refresh manually: `python refresh_auth.py`

Problem: Jobs not posting to Discord
Solution: Check channel IDs in `config.json`

Problem: Duplicate jobs appearing
Solution: Clear database: `python manage.py clear_seen_jobs`

6. Maintenance

- Updating tracked URLs
- Monitoring bot health
- Log file locations
- Backup procedures

Video Demo Requirements (Loom/Recording)

Content Structure (5-10 minutes):

1. **Introduction** (30 seconds)
 - Your name
 - Project overview
2. **Feature Walkthrough** (3 minutes)
 - Open Notepad, list all features
 - Explain each feature
3. **Live Demo** (5 minutes)
 - Show bot running
 - Demonstrate job posting in real-time
 - Show thread creation

- Display multiple URL support
- Demonstrate token refresh (optional)

4. **Code Walkthrough** (2 minutes)

- Project structure
 - Key files and their purpose
 - Highlight interesting solutions
-

Success Guidelines

Development Best Practices

Code Quality Standards:

```
# ■ GOOD
class UpworkScraper:
    """
    Handles job scraping from Upwork using HTTP requests.

    Attributes:
        session: HTTP session with persistent cookies
        auth_manager: Handles token refresh
    """

    def fetch_jobs(self, search_query: str) -> List[Dict]:
        """
        Fetch jobs matching search query.

        Args:
            search_query: Keywords to search for

        Returns:
            List of job dictionaries

        Raises:
            AuthenticationError: If tokens are invalid
        """
        pass

# ■ BAD
class scraper:
    def get(self, q):
        r = requests.post(url, data=d)
        return r.json()
```

Configuration Management:

```
# ■ GOOD - Environment variables
import os
from dotenv import load_dotenv

load_dotenv()

DISCORD_TOKEN = os.getenv("DISCORD_TOKEN")
UPWORK_EMAIL = os.getenv("UPWORK_EMAIL")

# ■ BAD - Hardcoded
DISCORD_TOKEN = "MTIzNDU2Nzg5MDEyMzQ1Njc4.GHij12.K3lM4n05pQ6rS7tU8vW9xY0zA1bC2dE3fG4h"
UPWORK_EMAIL = "myemail@example.com"
```

Error Handling:

```
# ■ GOOD
try:
    jobs = fetch_jobs(search_query)
except RequestException as e:
    logger.error(f"Failed to fetch jobs: {e}", exc_info=True)
    # Retry or handle gracefully
except AuthenticationError:
    logger.warning("Auth expired, refreshing...")
    refresh_tokens()
    jobs = fetch_jobs(search_query)

# ■ BAD
try:
    jobs = fetch_jobs(search_query)
except:
    pass
```

Questions to Guide Your Development

Phase 1 - Reverse Engineering:

- What network requests does Upwork make when I search for jobs?
- What's the difference between the search endpoint and details endpoint?
- Which headers are absolutely required vs optional?
- What happens if I remove certain cookies?

Phase 2 - Discord Integration:

- How do I format messages for readability?
- What's the API limit for Discord message length?
- Can I create threads programmatically?

- How do I handle rate limiting?

Phase 3 - Authentication:

- Where exactly are tokens stored (cookies vs localStorage)?
- How can I decode a JWT to check expiration?
- Can I refresh without full login?
- What's the minimum browser interaction needed?

Phase 4 - Multi-URL:

- How do I prevent posting the same job twice?
- Should I process URLs sequentially or concurrently?
- How do I validate that jobs match their source URL?
- What database structure makes sense?

Phase 5 - Production:

- What could cause memory leaks?
 - How do I test for 100 hours without waiting?
 - What metrics should I monitor?
 - How do I handle unexpected shutdowns?
-

Important Reminders

Network Requirements

■ **CRITICAL:** You **cannot** use **Bitbash Internet** for this project.

Options:

1. Use your own mobile hotspot
2. Use VPN with Bitbash internet
3. Use home internet connection

Why: Upwork may block requests from data center IPs or shared networks.

Communication Protocol

Do Ask:

- Questions about requirements
- Clarifications on acceptance criteria
- Help understanding concepts
- Feedback on approach

Don't Ask:

- "What's the exact GraphQL query?"
- "Can you write the token refresh code?"
- "What headers do I need?"
- "Can you debug my code?"

Time Management

Reverse engineering, basic HTTP scraping
Discord integration
Authentication management
Multi-URL support
Production hardening, documentation
Testing, video demo, final polish
100-hour stability test

Learning Resources (Hints Only)

Hint for Dev tools

1 - <https://www.loom.com/share/8c511b9be356412d96a655af9634f2e4?sid=71889bab-cD80-40a9-a8fe-c76df46a6971>

Recommended Reading

- **GraphQL:** graphql.org/learn
- **Browser Fingerprinting:** fingerprintjs.com/blog
- **JWT Tokens:** jwt.io
- **Discord.py:** discordpy.readthedocs.io
- **Async Python:** realpython.com/async-io-python

Tools Youll Need

- **Chrome DevTools:** For network analysis
- **Postman/Insomnia:** For testing API requests
- **DB Browser for SQLite:** For database inspection
- **htop/Activity Monitor:** For resource monitoring

Debugging Techniques

When stuck on API requests:

1. Compare your request to browser request side-by-side
2. Use `curl` to test outside Python
3. Check if cloudscraper vs requests makes a difference
4. Verify SSL/TLS settings

When stuck on authentication:

1. Clear all cookies and see what regenerates
 2. Check browser console for errors
 3. Look at Application tab for localStorage
 4. Use browser extension to export cookies
-

Final Words

This project will challenge you. It's **designed** to be difficult because it simulates real production problems:

- Working with undocumented APIs
- Handling authentication at scale
- Building resilient systems
- Making architectural decisions

Remember:

- **Understanding** is more important than speed
- **DevTools** is your most powerful tool
- **Document** your thought process

- **Persist** through challenges
- **Communicate** blockers early

The goal isn't perfection - it's demonstrating your ability to:

1. **Reverse engineer** complex systems
2. **Think critically** about architecture
3. **Build resilient** production code
4. **Learn independently** from observation

Good luck! You've got this.